Artificial Intelligence
Assignment Report 2

Warren Roberts
Student ID: 100450116
Student Email: wlr006@uark.edu
University of Arkansas
0 Days late used

Abstract

This report documents my implementation of Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), and A* Search in the UC Berkeley Pacman framework using Python 3.8+. The primary objective was to reach the food (goal state) while also satisfying the additional constraint that Pacman must hit a wall at least once but no more than twice. Each algorithm was tested on tiny, medium, and big mazes, and their node expansions and behavior under the framework's cost rules were compared.

Introduction

This assignment used the `PositionSearchProblem` interface, which provides `get_start_state()`, `is_goal_state(state)`, `is_wall(state)`, and `get_successors(state)`. Each search function returns a list of actions (North, South, East, West) that moves Pacman from the start position to the food.

The unique aspect of this homework was the wall-hit requirement: Pacman must hit a wall at least once, but no more than two times. Normally, search algorithms are designed to avoid illegal moves entirely, so this constraint required intentional handling outside the standard search logic.

Wall-Hit Strategy

In my implementation, the search algorithms themselves do **not** expand into walls. I filter successors using `not problem.is_wall(next_state)` to ensure the search tree only contains legal states. To satisfy the wall-hit requirement, I implemented a helper function called `first_hit(problem, state)` and prepend its actions to the final solution path.

The `first_hit` function checks the four adjacent cells around the current state. If a wall is adjacent, it returns two actions:

- The action that attempts to move into the wall (which counts as the wall hit),
- The opposite action to immediately return to the original position.

The final returned plan has the structure:

$$[hitwall, return] + (normalsearchpath)$$

This approach satisfies the constraint in a controlled and predictable way without modifying the core search implementations.

Depth-First Search (DFS)

DFS was implemented using a `util.Stack` (LIFO). It explores deeply along one branch before backtracking. I tracked visited states using a set to prevent cycles and infinite loops.

DFS does not guarantee the shortest path, but it demonstrates how search order significantly affects behavior. In larger mazes, DFS often follows long corridors before exploring alternative paths.

DFS Testing

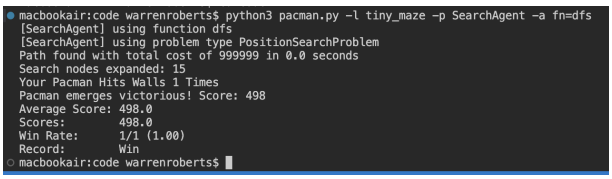

Figure 1: DFS on tiny_maze

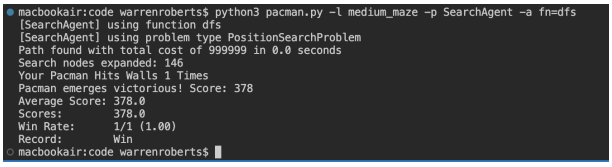

Figure 2: DFS on medium_maze

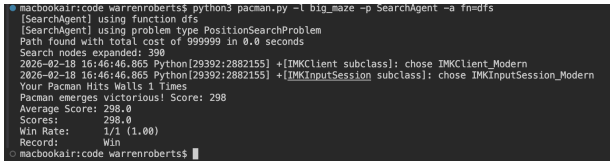

Figure 3: DFS on big_maze

Breadth-First Search (BFS)

BFS was implemented using a `util.Queue` (FIFO). It expands states level-by-level, which guarantees the shortest path in number of steps when all step costs are uniform.

Compared to DFS, BFS typically expands more nodes in larger mazes because it explores the state space systematically. However, it guarantees an optimal solution in terms of step count under uniform costs.

BFS Testing

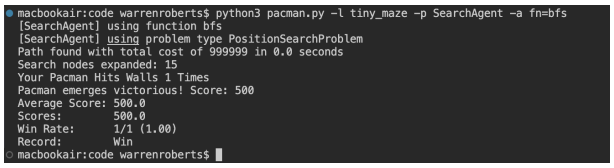

Figure 4: BFS on tiny_maze

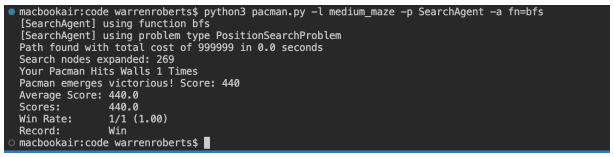

Figure 5: BFS on medium_maze

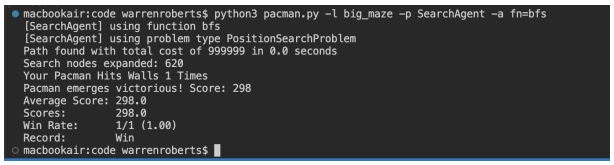

Figure 6: BFS on big_maze

Uniform Cost Search (UCS)

UCS was implemented using a `util.PriorityQueue` ordered by cumulative cost $g(n)$. I maintained a dictionary storing the lowest known cost to each state to prevent expanding higher-cost duplicate paths.

In this project's basic position search, all step costs are uniform. As a result, UCS behaves identically to BFS in terms of node expansions, which matches the results observed during testing.

The framework reports a cost of 999999 if the returned action list contains any illegal move. Since my final solution intentionally includes a wall hit, this cost output is expected and does not indicate an implementation error.

UCS Testing

Figure 7: UCS on tiny_maze

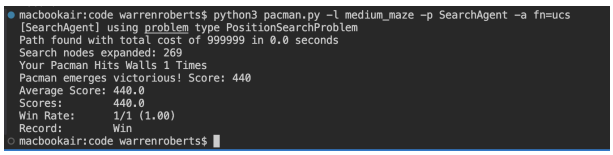

Figure 8: UCS on medium_maze

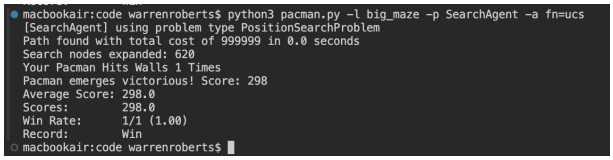

Figure 9: UCS on big_maze

A* Search

A* Search was implemented using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ represents the cost so far and $h(n)$ represents the heuristic estimate to the goal. A priority queue ordered by $f(n)$ was used, and the lowest known $g(n)$ values were tracked in a dictionary to ensure optimality.

Manhattan Heuristic

The Manhattan heuristic is defined as:

$$h(n) = |x - x_g| + |y - y_g|$$

This heuristic is admissible for grid-based movement without diagonals because it never overestimates the true shortest path.

Testing showed that A* with the Manhattan heuristic expanded fewer nodes than BFS and UCS on medium and big mazes, demonstrating the benefit of heuristic guidance.

A* Testing (Manhattan)

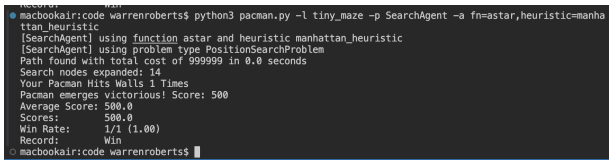

Figure 10: A* (Manhattan) on tiny_maze

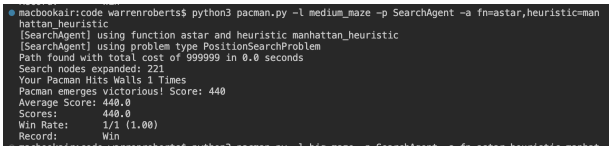

Figure 11: A* (Manhattan) on medium_maze

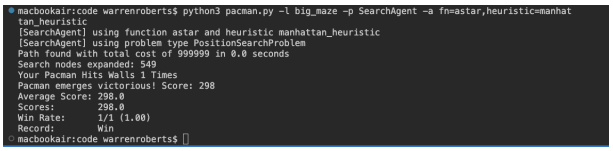

Figure 12: A* (Manhattan) on big_maze

Custom Heuristic

My custom heuristic computes both Manhattan and Euclidean distances and returns their maximum:

$$h(n) = \max(Manhattan, Euclidean)$$

In grid movement without diagonals, Manhattan distance is always greater than or equal to Euclidean distance. Therefore, this heuristic behaves identically to Manhattan in practice. This explains why node expansions were identical during testing.

Custom Heuristic Code

```python
def your_heuristic(state, problem=None):
    """ Your Custom Heuristic """
    x, y = state
    gx, gy = problem.goal

    # logic from search_agents.py
    manhattan_heuristic = abs(x - gx) + abs(y - gy)
    euclidean_heuristic = ((x - gx) ** 2 + (y - gy) ** 2) ** 0.5

    return max(manhattan_heuristic, euclidean_heuristic)
```

Figure 13: Custom heuristic implementation

A* Testing (Custom Heuristic)

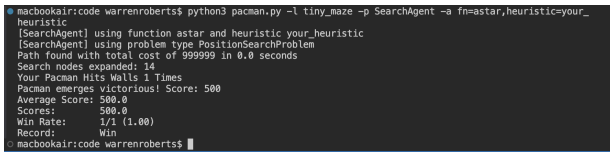

Figure 14: A* (Custom Heuristic) on tiny_maze

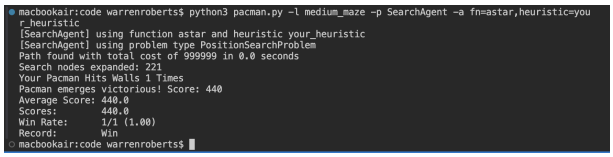

Figure 15: A* (Custom Heuristic) on medium_maze

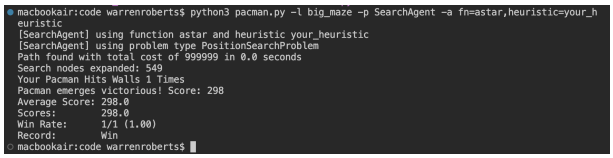

Figure 16: A* (Custom Heuristic) on big_maze

Results Summary

Across medium and big mazes, DFS expanded the most nodes due to deep exploration without cost awareness. BFS and UCS expanded the same number of nodes because step costs were uniform. A* consistently expanded fewer nodes when using the Manhattan heuristic, confirming the benefit of heuristic guidance.

Conclusion

DFS, BFS, UCS, and A* were successfully implemented and reached the goal in tiny, medium, and big mazes while satisfying the wall-hit constraint (one wall hit in my runs). BFS and UCS expanded the same number of nodes due to uniform step costs. A* expanded fewer nodes than BFS and UCS, confirming that the heuristic effectively guides the search toward the goal.

The wall-hit constraint was handled cleanly by prepending a controlled two-action sequence using `first_hit`, rather than altering the internal search logic or expanding illegal states.